



INSTITUTO TECNOLÓGICO DE TEHUACÁN

REPORTE DE RESIDENCIA PROFESIONAL

EXPLORACIÓN DEL ESPACIO LATENTE PARA LA IDENTIFICACIÓN DE ANOMALÍAS EN REDES

SANTIAGO DOMÍNGUEZ AGUSTÍN

INGENIERÍA EN SISTEMAS COMPUTACIONALES

ASESOR INTERNO: FERNANDO CANSINO GÁLVEZ

ASESOR EXTERNO: DR. JOSÉ ROBERTO PÉREZ CRUZ

FECHA: 14 DE DICIEMBRE DE 2025

Índice general

CAPÍTULO

1. GENERALIDADES DEL PROYECTO	1
1.1. Introducción	1
1.2. Descripción de la Empresa	2
1.2.1. Datos generales	2
1.2.2. Misión	2
1.2.3. Visión	2
1.2.4. Valores fundamentales	2
1.2.5. Organigrama	3
1.2.6. Descripción del área	3
1.3. Problemas a resolver	4
1.4. Objetivos	4
1.4.1. Objetivo general	4
1.4.2. Objetivos específicos	4
1.5. Justificación	5
2. MARCO TEÓRICO	7
2.1. Métodos y definiciones	7
2.1.1. Conceptos de estadística	7
2.1.1.1. Media aritmética	7
2.1.1.2. Varianza	8
2.1.1.3. Desviación estándar	8
2.1.1.4. Distribución normal	9
2.1.1.5. Regla empírica	10
2.1.2. Conceptos de <i>Machine Learning</i>	10
2.1.2.1. Distancia euclidiana	10
2.1.2.2. Clustering	11
2.1.2.3. Algoritmo k-means	11
2.1.2.4. Suma de los errores cuadrados (SSE)	12
2.1.2.5. Método del codo	12
2.1.3. Conceptos de ciberseguridad	13

2.1.3.1. Ataques cibernéticos	13
2.1.4. Ataques de denegación de servicio	13
2.1.4.1. Ataque Smurf	13
2.1.4.2. Ataque Neptune	14
2.1.5. Ataques de reconocimiento	14
2.1.5.1. Scanning	14
2.1.6. Ataques de escalamiento de privilegios	15
2.1.6.1. R2L	15
2.1.6.2. U2R	15
2.1.7. Sistema de detección de intrusiones	16
2.1.8. Datasets	16
2.1.8.1. KDDCUP99	16
2.1.8.2. NSL-KDD	17
2.2. Antecedentes (Estado del arte)	17
2.2.1. Contexto histórico	17
2.2.2. Adaptabilidad y grandes cantidades de datos	17
2.2.3. Evolución hacia los modelos híbridos	18
2.2.4. La amenaza de los ataques DDoS	18
2.2.5. Los nuevos retos	19
3. DESARROLLO	21
3.1. CRONOGRAMA DE ACTIVIDADES	21
3.2. Preparación del entorno de trabajo	23
3.2.1. Análisis de distintos IDE	23
3.2.2. Creación de un entorno virtual	23
3.2.3. Instalación y configuración de Python	23
3.2.4. Exploración y comprensión de las características	24
3.3. Código k-means	24
3.4. Limpieza y procesamiento de los datos	25
3.4.1. Carga de KDDCUP99	25
3.4.2. Eliminación de características innecesarias	25
3.5. k-means con dos clústeres	25
3.6. Desbalance de datos	26
3.7. Método del codo para el k óptimo	27
3.8. Método del codo con datos normalizados	27
3.9. Modernización del código y Optimización de la Arquitectura de Datos	28
3.9.1. Migración de RDDs por dataframes	28

3.9.2. Actualización de librerías	28
4. RESULTADOS	29
4.1. k-means con dos clústeres	30
4.2. Desbalance de datos	31
4.3. Método del codo para el k óptimo	32
4.4. Método del codo con datos normalizados	33
4.5. Clústeres antes de la normalización	34
4.6. Clústeres después de la normalización	35
5. CONCLUSIONES	37
5.1. Recomendaciones y trabajo futuro	38
A. CÓDIGO FUENTE DEL PROYECTO	41

GENERALIDADES DEL PROYECTO

1.1. Introducción

La ciberseguridad en la red siempre ha sido de suma importancia. Desde la creación del internet han existido todo tipo de ataques que se distribuyen mediante la red. Ante esto surgieron los IDS (Sistemas de Detección de Intrusiones) los cuales han evolucionado a la par de los ataques cibernéticos. Diana et al. (2025)

Ante esta situación de amenazas constantes y cada vez más sofisticadas, los mecanismos tradicionales de seguridad resultan ineficientes contra ataques desconocidos. Por ello, surgió la necesidad de implementar soluciones más inteligentes que permitan la detección de intrusiones en tiempo real, como lo es la técnica de machine learning (ML) en los IDS ya que posibilita la detección de patrones sin la necesidad de firmas previas, lo que mejora la capacidad de respuesta de ataques nuevos.

En el trabajo *Exploración del espacio latente para la identificación de anomalías en redes* se plantea confrontar a los ataques más modernos sin necesidad de que los conozca.

Como una parte fundamental se realizó un estudio comparativo con el dataset KDDCUP99, basado en el algoritmo de agrupamiento k-means. Este se implementó dentro de un entorno virtual con Python y las últimas librerías de PySpark, que permitieron su implementación dentro del entorno moderno de Python. A lo largo del documento se detalla el proceso que se ha realizado como la limpieza del dataset, el número óptimo de clústeres con el método del codo y la normalización para la interpretación final de las gráficas. El documento se ha hecho de tal forma que también pueda replicarse.

Por último, en los resultados se muestran las interpretaciones de todas las gráficas en las que se pueden visualizar las anomalías dentro de la red del dataset KDDCUP99.

1.2. Descripción de la Empresa

1.2.1. Datos generales

Nombre de la empresa: Instituto Nacional de Astrofísica, Óptica y Electrónica (INAOE)

Giro: El giro principal del INAOE (Instituto Nacional de Astrofísica, Óptica y Electrónica) es la investigación y el desarrollo tecnológico en sus áreas de especialización, junto con la formación académica de posgrado, la vinculación con la industria y la sociedad, y la realización de trámites gubernamentales.

Dirección: Calle Luis Enrique Erro #1, colonia Santa María Tonantzintla, San Andrés Cholula, C.P. 72840, México, Puebla.

Teléfono: (222) 266 31 00

Correo electrónico: difusion@inaoep.mx

1.2.2. Misión

Generar conocimiento a través de la investigación científica, desarrollo tecnológico, y formación de recursos humanos, así como vincularse con la sociedad para aplicar dichos conocimientos, y recursos humanos, en la creación de bienestar social nacional e internacional.

1.2.3. Visión

Ser un centro de investigación, desarrollo tecnológico, y formación de recursos humanos de vanguardia, y calidad certificada, a nivel nacional e internacional en las disciplinas de astrofísica, óptica, electrónica, ciencias computacionales, y áreas afines. Asimismo, ser un centro que coadyuve a la integración de la cadena de valor desde la ciencia básica hasta el desarrollo tecnológico, y su posterior vinculación con la sociedad para propiciar que la investigación y el desarrollo tecnológico sean el motor de desarrollo de México.

1.2.4. Valores fundamentales

Se centran en el servicio al Bien Común, la Justicia (apego a la ley), la Generosidad (solidaridad con grupos vulnerables), y la Igualdad, buscando el desarrollo integral de la sociedad y sus miembros, reflejados en su Código de Conducta

como marco para servidores públicos que deben servir a la ciudadanía con integridad y respeto al Estado de Derecho.

1.2.5. Organigrama

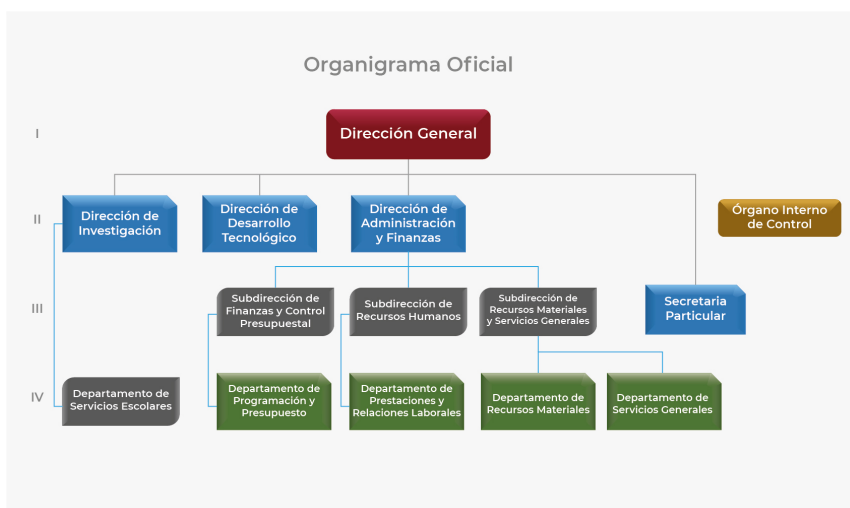


Figura 1.1: Organigrama oficial del INAOE

1.2.6. Descripción del área

En 1972 se fundó el Departamento de Óptica, y dos años después inició sus actividades el Departamento de Electrónica. Desde su creación uno de los principales objetivos del INAOE ha sido la preparación de investigadores jóvenes, capaces de identificar y resolver problemas científicos y tecnológicos en astrofísica, óptica, electrónica y áreas afines. En 1972 se iniciaron los estudios de maestría en Óptica y en 1974 los de Electrónica. En 1984 se inició el programa de doctorado en Óptica, y en 1993 los programas de doctorado en Electrónica; así como la maestría y doctorado en Astrofísica. Finalmente, en agosto de 1998 se inició el programa de maestría y doctorado en Ciencias Computacionales.

El INAOE, como Centro Público de Investigación contribuye al desarrollo científico y tecnológico del país mediante la formación de investigadoras/es de alto

nivel, capaces de adquirir, generar y aplicar conocimientos en el área de Tecnologías de Seguridad Informática, específicamente en los campos de: criptografía, vigilancia de la red, internet de las cosas, confianza cero, Blockchain y normatividad. Estos recursos humanos deberán contribuir al desarrollo de los sectores social, educativo y productivo en la región y en el país.

1.3. Problemas a resolver

La necesidad de formar personas capacitadas en las tecnologías de la seguridad informática que puedan actuar en contra de los problemas actuales, como la vigilancia de redes, protección de los datos, la confianza cero, el IoT y el cumplimiento normativo para fortalecer la seguridad.

1.4. Objetivos

1.4.1. Objetivo general

Realizar un estudio comparativo (survey) de los principales datasets públicos utilizados para el análisis de tráfico de red, detección de anomalías, ataques cibernéticos y evaluación de sistemas de detección de intrusos (IDS). Este estudio servirá como base para el diseño y validación del corpus de datos que será recolectado en el entorno virtualizado propuesto en la Etapa I.

1.4.2. Objetivos específicos

- Analizar el estado del arte del conjunto de datos estandarizado KDDCUP99 y los distintos modelos de los IDS analizando sus fortalezas y debilidades.
- Actualizar la implementación del algoritmo k-means en el entorno de PySpark.
- Actualizar las librerías del código k-means
- Documentar el proceso de la actualización

1.5. Justificación

La detección de amenazas en redes de cómputo continúa siendo un reto fundamental en el ámbito de la ciberseguridad, particularmente ante la creciente complejidad, volumen y heterogeneidad del tráfico de red. En este contexto, los enfoques de aprendizaje no supervisado resultan especialmente relevantes, ya que permiten identificar comportamientos anómalos sin depender de etiquetas previamente definidas, las cuales suelen ser incompletas, costosas de obtener o rápidamente obsoletas frente a nuevas variantes de ataque.

El dataset KDDCUP99 constituye un referente clásico en la investigación sobre detección de intrusiones y ha sido ampliamente utilizado como base para el desarrollo y evaluación de modelos de detección de anomalías. No obstante, una parte significativa de los pipelines y herramientas existentes para su análisis fueron desarrollados originalmente en Python 2, lenguaje que ha quedado oficialmente discontinuado, lo que limita su uso, mantenimiento e integración con infraestructuras modernas de análisis de datos y procesamiento distribuido.

En este trabajo se desarrolló la actualización de un pipeline de detección de amenazas basado en k-means, mediante la migración del código a Python 3, garantizando su compatibilidad con entornos actuales. Asimismo, se realizó una reestructuración de las representaciones de datos, sustituyendo estructuras tradicionales por DataFrames, lo que permitió mejorar la claridad, mantenibilidad y eficiencia del proceso de análisis.

Adicionalmente, se actualizaron las dependencias de PySpark, asegurando la correcta ejecución del pipeline en plataformas modernas de procesamiento distribuido y facilitando su escalabilidad hacia escenarios de análisis de grandes volúmenes de datos. En conjunto, el trabajo realizado contribuye a la modernización de herramientas clásicas de detección de intrusiones y establece una base técnica sólida para su reutilización y extensión en contextos actuales de ciberseguridad y analítica de datos.

CAPÍTULO 2

MARCO TEÓRICO

2.1. Métodos y definiciones

2.1.1. Conceptos de estadística

La estadística proporciona el conjunto de herramientas matemáticas necesarias para describir, analizar e interpretar datos. Su aplicación es fundamental en el desarrollo de modelos de aprendizaje automático, especialmente en tareas de detección de anomalías, donde resulta indispensable comprender la distribución, dispersión y estructura de los datos. En términos generales la estadística se encarga de la recolección, la manipulación y el tratamiento de información o datos para que puedan interpretarse y a partir de ellos tomar decisiones. (Perez, 2023, p. 19)

2.1.1.1. Media aritmética

La media aritmética es una medida de tendencia central que describe el valor representativo de un conjunto de datos. Permite resumir el comportamiento típico de una variable cuantitativa y resulta fundamental en análisis exploratorios y en la caracterización de patrones normales en conjuntos de datos.

Definición 2.1 (Media aritmética). Dada una muestra de n observaciones numéricas $x_1, x_2, \dots, x_n$, la media aritmética se define como:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad (2.1)$$

donde $\bar{x}$ representa el valor promedio de la muestra.

La media sirve como punto de referencia para cuantificar la magnitud de las desviaciones individuales y, en consecuencia, para apoyar la identificación de comportamientos atípicos.

En el análisis de tráfico de red, la media se utiliza para obtener valores promedio de métricas como la duración de conexiones, el número de paquetes enviados, la cantidad de bytes transferidos o las tasas de flujo. Estas medidas permiten establecer líneas base de comportamiento normal, a partir de las cuales se pueden detectar variaciones abruptas asociadas a posibles actividades maliciosas. Es común referirse a la media como *promedio* (ILLOWSKY, 2023, p. 120).

2.1.1.2. Varianza

La varianza es una medida de dispersión que cuantifica qué tanto se alejan los valores de un conjunto de datos respecto a su media. Mientras que la media describe el valor típico o central, la varianza indica el grado de variabilidad presente en los datos, lo cual resulta fundamental para analizar la estabilidad de un sistema y detectar posibles valores atípicos.

Definición 2.2 (Varianza muestral). Sea una muestra de n observaciones $x_1, x_2, \dots, x_n$ con media muestral $\bar{x}$. La varianza muestral se define como:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2 \quad (2.2)$$

donde s^2 representa la variabilidad promedio de los datos respecto a la media.

Valores altos de s^2 indican una mayor dispersión en las observaciones, mientras que valores bajos reflejan que los datos se agrupan más cercanamente alrededor de $\bar{x}$.

En el análisis de tráfico de red, una varianza elevada en métricas como el número de paquetes, la duración de las conexiones o los bytes transferidos puede indicar comportamientos inestables o cambios bruscos en el patrón normal de operación. Estas fluctuaciones pueden estar asociadas a eventos anómalos o potencialmente maliciosos, por lo que la varianza se convierte en una herramienta clave para apoyar la detección de anomalías (ILLOWSKY, 2023, p. 120).

2.1.1.3. Desviación estándar

La desviación estándar es una medida de dispersión que indica qué tanto se alejan los valores de un conjunto de datos respecto a su media, pero expresada en las mismas unidades que los datos originales. A diferencia de la varianza, que utiliza desviaciones cuadráticas, la desviación estándar facilita la interpretación

directa de la variabilidad, lo que la convierte en una herramienta ampliamente utilizada en estadística aplicada y análisis de datos.

Definición 2.3 (Desviación estándar muestral). Dada una muestra de n observaciones $x_1, x_2, \dots, x_n$ con varianza muestral s^2 , la desviación estándar muestral se define como:

$$s = \sqrt{s^2} = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2} \quad (2.3)$$

donde s cuantifica la dispersión promedio de las observaciones respecto a la media $\bar{x}$.

Valores grandes de s indican mayor variabilidad en los datos, mientras que valores pequeños reflejan una distribución más compacta alrededor de la media.

Desviaciones estándar inusualmente altas pueden sugerir comportamientos irregulares, como ráfagas de tráfico, conexiones anómalas o patrones asociados a ataques de denegación de servicio, lo que la convierte en un indicador fundamental para la detección de anomalías (ILLOWSKY, 2023, p. 120).

2.1.1.4. Distribución normal

La distribución normal, también conocida como distribución gaussiana, es una de las distribuciones de probabilidad más importantes en estadística debido a su presencia en numerosos fenómenos naturales y al papel central que desempeña en diversos métodos matemáticos y de aprendizaje automático. Su forma simétrica y concentrada alrededor de la media permite modelar comportamientos típicos y variaciones esperadas en un conjunto de datos, lo cual es fundamental para determinar qué valores pueden considerarse anómalos.

Definición 2.4 (Distribución normal). Una variable aleatoria continua X sigue una distribución normal con media μ y desviación estándar σ si su función de densidad de probabilidad está dada por:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \quad (2.4)$$

lo que se denota como $X \sim N(\mu, \sigma)$. Cuando $\mu = 0$ y $\sigma = 1$, la variable sigue una *distribución normal estándar*, denotada como $X \sim N(0, 1)$.

En el contexto del análisis de tráfico de red, la distribución normal permite modelar el comportamiento típico de ciertas métricas que presentan oscilaciones alrededor de un valor promedio, como el tiempo entre paquetes, la variación en el tamaño de los flujos o la duración de sesiones.

2.1.1.5. Regla empírica

La regla empírica describe la proporción de valores que se encuentran a una, dos o tres desviaciones estándar de la media cuando los datos siguen una distribución normal. Esta regla permite establecer criterios simples para identificar valores atípicos basados en su distancia respecto al comportamiento esperado.

Definición 2.5 (Regla empírica). Si una variable aleatoria X sigue una distribución normal con media μ y desviación estándar σ , entonces se cumple que:

- Aproximadamente el 68 % de los valores se encuentran en el intervalo $\mu \pm 1\sigma$.
- Aproximadamente el 95 % se encuentran en el intervalo $\mu \pm 2\sigma$.
- Aproximadamente el 99.7 % se encuentran en el intervalo $\mu \pm 3\sigma$.

En detección de anomalías, estos rangos permiten establecer umbrales simples para identificar valores extremos que podrían indicar comportamientos maliciosos o inusuales.

2.1.2. Conceptos de *Machine Learning*

2.1.2.1. Distancia euclidiana

La distancia euclidiana es una medida de disimilitud entre dos puntos en un espacio cartesiano. Se utiliza ampliamente en algoritmos de clasificación y agrupamiento para determinar qué tan similares o diferentes son dos observaciones dentro de un conjunto de datos.

Definición 2.6 (Distancia euclidiana). Sean dos puntos en $\mathbb{R}^d$, $X = (x_1, x_2, \dots, x_d)$ y $Y = (y_1, y_2, \dots, y_d)$. La distancia euclidiana entre ellos se define como:

$$d(X, Y) = \sqrt{\sum_{i=1}^d (x_i - y_i)^2} \quad (2.5)$$

Esta distancia cuantifica la separación geométrica entre los puntos.

En detección de anomalías, distancias elevadas entre una observación y los centroides de sus clústeres pueden indicar comportamientos fuera de lo normal en el tráfico de red.

2.1.2.2. Clustering

El clustering es una técnica no supervisada que organiza los datos en grupos (clústeres) de tal manera que los elementos dentro de un mismo grupo sean más similares entre sí que con elementos de otros grupos. Esta técnica es fundamental en la detección de anomalías, donde los puntos alejados de cualquier clúster suelen considerarse sospechosos.

Definición 2.7 (Clustering). El clustering consiste en la partición de un conjunto de datos en k grupos disjuntos $C_1, C_2, \dots, C_k$ tales que:

$$\bigcup_{j=1}^k C_j = X, \quad C_i \cap C_j = \emptyset \quad \text{si } i \neq j, \quad (2.6)$$

donde los elementos dentro de cada clúster poseen una alta similitud de acuerdo con una métrica definida (generalmente la distancia euclidiana).

En análisis de tráfico de red, los clústeres pueden captar patrones típicos de comportamiento, mientras que puntos alejados pueden corresponder a conexiones anómalas o ataques.

2.1.2.3. Algoritmo k-means

El algoritmo k-means es una técnica de aprendizaje no supervisado utilizada para agrupar datos en k clústeres basándose en su similitud. Su objetivo es particionar los datos de tal forma que cada observación pertenezca al clúster cuyo centroide se encuentre más cercano, minimizando la variabilidad interna de los grupos.

Definición 2.8 (Algoritmo k-means). Dado un conjunto de datos $\{x_1, x_2, \dots, x_n\} \subset \mathbb{R}^d$ y un número de clústeres k , el algoritmo k-means busca los centroides $\mu_1, \dots, \mu_k$ que minimizan la función objetivo:

$$\mathcal{J} = \sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - \mu_j\|^2, \quad (2.7)$$

donde C_j es el conjunto de puntos asignados al clúster j .

k-means es ampliamente utilizado en detección de anomalías porque las observaciones alejadas de los centroides o asignadas a clústeres pequeños pueden corresponder a comportamientos atípicos o potencialmente maliciosos.

2.1.2.4. Suma de los errores cuadrados (SSE)

La SSE mide la compacidad de los clústeres formados por un algoritmo como k-means. Es una métrica fundamental para evaluar la calidad del agrupamiento y para determinar el número de clústeres más adecuado.

Definición 2.9 (Suma de los errores cuadrados (SSE)). Dado un conjunto de clústeres $C_1, C_2, \dots, C_k$ con centroides $\mu_1, \mu_2, \dots, \mu_k$, la SSE se define como:

$$\text{SSE} = \sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - \mu_j\|^2 \quad (2.8)$$

donde $\|x_i - \mu_j\|^2$ es la distancia cuadrática entre cada punto y su centroide.

Valores bajos de SSE indican grupos compactos y bien definidos; valores altos sugieren dispersión o mala estructura, lo cual es clave para identificar ruido o anomalías.

2.1.2.5. Método del codo

El método del codo es una técnica visual para determinar un número adecuado de clústeres en algoritmos como k-means. Evalúa la SSE para diferentes valores de k y selecciona el punto donde la mejora comienza a disminuir drásticamente.

Definición 2.10 (Método del codo). El método del codo consiste en calcular la SSE para valores de $k = 1, 2, \dots, K$ y graficar la curva:

$$k \mapsto \text{SSE}(k) \quad (2.9)$$

El número óptimo de clústeres k^* se identifica en el punto donde la disminución de SSE empieza a ser marginal, formando un *codo* en la gráfica.

Este método permite elegir un número de clústeres que balancea precisión y simplicidad, evitando sobreajuste y mejorando la detección de patrones anómalos.

2.1.3. Conceptos de ciberseguridad

2.1.3.1. Ataques cibernéticos

Un ataque es una acción deliberada tomada contra un objetivo para afectar la confidencialidad, integridad, disponibilidad o autenticidad del sistema. Los ataques pueden ser activos o pasivos y pueden iniciarse desde dentro o fuera de la organización donde se encuentre la red.

Siempre ha sido un problema difícil de detectar porque cada vez desarrollan distintos tipos de ataques por lo que dejan de funcionar los métodos antiguos (firmas), como respuesta, se desarrollaron otros métodos para enfrentar este problema.

Estos ataques se pueden identificar como comportamientos anómalos ya que presentan patrones que no encajan con los comportamientos normales dentro del tráfico de red (Sheikh, 2021, p. 3).

2.1.4. Ataques de denegación de servicio

Un ataque de denegación de servicio (Denial of Service o DoS) es una forma de hacer que un servicio deje de estar disponible, negando así el acceso a los usuarios. A menudo, esto se logra bloqueando los recursos necesarios para proveer el servicio. Una de las formas más efectivas de hacerlo es generar numerosas solicitudes falsas desde diferentes, o distribuidas, fuentes, lo que ahoga las solicitudes legítimas.

Este tipo de ataque es el más presente dentro del dataset KDDCUP99, debido a que siempre se presenta en enormes cantidades, siendo uno de los menos complicados de detectar y menos perjudiciales en cuanto a la seguridad de los datos (Chou, 2018, p. 86).

2.1.4.1. Ataque Smurf

El ataque se aprovecha de una característica del protocolo ICMP, este esta hecho para siempre responder mensajes, la forma más común en la que solemos verlo es para realizar un ping. Así que el ataque consiste en el envío masivo de paquetes “*Echo request*” hacia la víctima.

Otra característica crucial es que al consistir en muchas conexiones y que no se realizan eso queda totalmente reflejado en las características del dataset `Srv_Count` y `Srv_error_rate`.

En otras palabras un ataque Smurf es cuando el atacante envía tráfico ICMP extra a direcciones de *broadcast* ip con una ip de origen spoofeada (falsificada) de la víctima (Sheikh, 2021, p. 86).

2.1.4.2. Ataque Neptune

Siendo un ataque que pertenece al tipo DoS comparten las mismas señales de ser ataques bastante abundantes así como lo es el ataque Smurf. Es el segundo ataque más frecuente dentro del dataset KDDCUP99 y se puede visualizar en las características con valores muy altos que son count y srv_count y con un valor fijo en src_bytes.

También conocidos como ataques TCP SYN semiabiertos, estos implican enviar paquetes SYN repetidos con direcciones IP falsas y puertos aleatorios a cada puerto en el servidor objetivo. Esto causa saturación de la red y resulta en un alto número de conexiones de error SYN en comparación con otros tipos de ataques (Bunmi, 2024, p. 83).

2.1.5. Ataques de reconocimiento

Es uno de los cuatro tipos de ataques presentes en la base de datos KDD-CUP99, este no se basa en atacar si no en una fase anterior al ataque siendo una de las partes más importantes en un ciberataque, aun así es posible detectarlo porque siguen siendo anomalías que no son parte del comportamiento normal de la red.

En la fase de reconocimiento, que es la fase de planificación, un atacante reúne tanta información como sea posible sobre el objetivo. La simple investigación puede ser la primera actividad en esta fase. El atacante puede luego pasar a otros métodos de reconocimiento, como el "dumpster diving"(revisar la basura) o el Scanning (Sheikh, 2021, p. 4).

2.1.5.1. Scanning

Durante la fase de escaneo, el atacante intenta identificar vulnerabilidades específicas. Los escáneres de vulnerabilidades son las herramientas más utilizadas. Los escáneres de puertos se utilizan para reconocer puertos a la escucha que proporcionan pistas sobre los tipos de servicios que se están ejecutando (Sheikh, 2021, p. 5).

2.1.6. Ataques de escalamiento de privilegios

Estos tipos de ataque dentro de KDDCUP99 son los más difíciles de detectar pues son los menos presentes y mejores ocultos pero también es de los más peligrosos ya que compromete a todo el sistema.

Existen usuarios maliciosos que desean tener acceso no solo a sus propios elementos sino también a los que están fuera de su contexto de permisos. Para ello podrían atacar a un programa vulnerable para obtener permisos de ejecución superiores a los que les corresponderían de manera normal. De lograrlo estarían consumando un ataque de escalación de privilegios sobre el sistema operativo (Vera, 2013, p. 8).

2.1.6.1. R2L

Este ataque se inicia desde el exterior pues para realizarse comienza intentando acceder de alguna a alguna cuenta de algún usuario con cualquier tipo de acceso, de modo que cuando ya consiga un acceso este hará un escalamiento de privilegios logrando vulnerar todo el sistema. El ataque se realiza buscando alguna vulnerabilidad, intentando adivinar la contraseña o engañar a algún usuario.

De la misma forma se puede observar mediante la estadística como datos fuera de lo común por lo que estas acciones se reflejan en un pequeño clúster alejado del resto, aunque esto se refleja en el proceso que intenta acceder a los usuarios, pues una vez que tiene acceso a uno, su comportamiento ya es muy parecido al de un usuario normal por lo que su detección se basa en detectarlo antes de lograr acceder.

De forma breve se conoce como obtener un acceso a una cuenta local desde una máquina remota (Frank, 2022, p. 16).

2.1.6.2. U2R

Este ataque, a diferencia del ataque R2L, no comienza desde el exterior así que es mucho más importante poder detectarlo mientras intenta realizar el escalamiento de privilegios porque cuando lo logra también su comportamiento es casi como el de un usuario normal.

Los ataques de red U2R son ataques al sistema en los que un *hacker* inicia un sistema con una cuenta de usuario normal e intenta abusar de las vulnerabilidades del sistema para obtener privilegios de súper usuario (Y.Mehelbei, 2022,

p. 19).

2.1.7. Sistema de detección de intrusiones

La detección de intrusiones es el proceso de monitorizar y analizar eventos en los sistemas o redes para identificar actividades maliciosas o no autorizadas.

Con respuesta a estos ataques, se propone un sistema de detección de intrusiones (IDS), la motivación del proyecto radica en que los IDS convencionales se basaban en firmas o patrones de ataques conocidos. En respuesta se propone el uso de machine learning y algoritmos de agrupamiento como solución más robusta (Coronado, 2024, p. 15).

2.1.8. Datasets

Es un conjunto de estructurado de datos que contiene información relacionada para su uso de forma eficaz, el cual sirve para poder hacer un análisis. En el caso de los IDS sirve de colección de instancias utilizadas para construir o evaluar el rendimiento de un modelo de aprendizaje automático.

De máxima importancia ya que son la base de los estudios de los ataques cibernéticos, pues estos guardan toda la información del tráfico de red por lo cual se pueden analizar los distintos comportamientos que tienen estos y de esta forma poder entender de que forma se puede actuar o entrenar modelos para la prevención de los ataques (Youdi, 2023, p. 2).

2.1.8.1. KDDCUP99

El conjunto de datos KDDCUP99 es uno de los más utilizados en la investigación de los IDS, ya que contiene información detallada de un entorno simulado con ataques de red y distintos comportamientos normales. Cada registro del KDDCUP99 representa una conexión de red entre las cuales se les puede clasificar como categóricos y numéricos que contienen las 41 características.

4 Categóricos: Protocolo de red, servicio de destino, estado de la conexión y etiqueta de la conexión.

37 Numéricos: Bytes enviados desde origen al destino, bytes enviados desde destino al origen, cantidad de paquetes con errores (flags de conexión)

- **Básicos:** Volumen de transferencia (bytes origen-destino) y duración.

- **Contenido:** Información dentro del paquete, como accesos a directorios o fallos de login.
- **Tráfico:** Estadísticas calculadas en una ventana de 2 segundos (ej. tasa de errores SYN/REJ, cantidad de conexiones al mismo nodo).

Es reconocido como el conjunto de datos de seguridad cibernética más frecuente y fue considerado un estándar de facto para las pruebas de referencia de IDS durante años. Siendo la base más importante en la historia para el estudio de los ataques cibernéticos con un dataset, por lo tanto es el dataset usado para el desarrollo de este proyecto (Frank, 2022, p. 16).

2.1.8.2. NSL-KDD

NSL-KDD mejora aún más la calidad de los datos al eliminar registros duplicados y disminuir el tamaño del conjunto de datos. "original de KDD'99"

Estos cambios hicieron que mejoraran los resultados en las pruebas de los distintos algoritmos de detección de intrusiones teniendo resultados más realistas que demostraban resultados no tan inflados (Frank, 2022, p. 16).

2.2. Antecedentes (Estado del arte)

2.2.1. Contexto histórico

El desarrollo de las redes ha estado intrínsecamente ligado a las amenazas de seguridad desde sus inicios. A medida que la tecnología se va volviendo accesible y su adopción crece, comienzan a ocurrir en aumento ataques cibernéticos cada vez más avanzados y de múltiples tipos. Esta problemática se le comenzó a dar una verdadera importancia a finales de los 90 ya que se desarrolló el dataset que fue el punto de referencia en 1999 el KDDCUP99 en un entorno de simulación militar. El cual sirvió como referencia para la creación de Sistemas de Detección de Intrusiones (IDS) y la evaluación comparativa (benchmarks) basados en algoritmos de aprendizaje. Este proceso de creación de los datasets y la mejora de modelos se ha estado repitiendo hasta la actualidad.

2.2.2. Adaptabilidad y grandes cantidades de datos

Las redes comenzaron a carecer mediante la nueva demanda y nuevas herramientas como la IA, así que como respuesta, una solución que surgió fueron

las Redes Definidas por Software (SDN) como un paradigma flexible para facilitar la gestión. A pesar de que existen los NIDS, los modelos de aprendizaje profundo (DL) son clave para la extracción automática de las características Sayed et al. (2022). A pesar de los avances en la arquitectura de la red, el entrenamiento de IDS enfrenta problemas como el volumen masivo (Big Data), ante esto se ha propuesto Sobremuestreo Aleatorio (RO) junto con incrustación de características demostrando que la calidad del preprocesamiento es tan vital como el modelo mismo Talukder et al. (2024).

2.2.3. Evolución hacia los modelos híbridos

Con el paso del tiempo se han propuesto varias soluciones como el Machine Learning (ML) y el Deep Learning (DL), sin embargo, por si solos tienen desventajas, como respuesta a la tendencia actual se dirige a modelos híbridos y multimodales. Así como el Modelo Multimodal Híbrido (MHPN) que imita el análisis manual de expertos al procesar la información actual que busca aprovechar la estadística de red para generar patrones generales y metadatos del flujo de datos. DAS (2024), mientras que otro MHPN muy parecido extrae las características de dos fuentes simultáneamente, la información estadística del tráfico y la carga útil original que es el payload, todo esto para alimentar al clasificador y gracias a todo esto da muy buenos resultados Shi et al. (2023). Para cubrir las deficiencias de la extracción de las características temporales se plantea el sistema DCNN-BILSTM que combina redes neuronales profundas con memoria a corto y largo plazo logrando una extracción robusta Hnamte y Hussain (2023). Otra de las estrategias es usar Doble Conjunto (Dual-IDS) que combinan técnicas de Bagging y Árboles de Decisión con Gradient Boosting (GBDT) para maximizar las ventajas de ambos simultáneamente Louk y Tama (2023).

2.2.4. La amenaza de los ataques DDoS

Por otro lado se han desarrollado IDS con propósitos más específicos como la detección de las amenazas de los ataques DDoS que tienen un gran impacto económico y operativo. Se han diseñado modelos basados específicamente en Deep Learning que han conseguido 99.30 % de efectividad en el dataset CIC-DDoS2019 Akgun et al. (2022). Otra forma de usar el DL usar el aprendizaje supervisado y no supervisado aprovechando que los ML fallan debido a su aprendizaje superficial así que como respuesta está el Autoencoder Contractivo

Profundo (DCAE), este modelo busca aprenderse un tráfico normal para detectar el que no lo es Aktar y Yasin Nur (2023).

2.2.5. Los nuevos retos

Actualmente se ha desatado una lucha de fuego contra fuego donde los propios modelos de ML/DL son vulnerables a Ataques Adversarios (Adversarial ML), ya que imitan el comportamiento normal para pasar desapercibidos. Se demostró una solución a los ataques DDoS generados es creando dos modelos diferentes, un discriminador para detectar el tráfico generado por IA y un IDS basado en LSTM para clasificar el tráfico real como normal o ataque Mustapha et al. (2023). En una solución más avanzada y general aborda la vulnerabilidad de los IDS basados Machine Learning. Con Apollon, un sistema de defensa de múltiples clasificadores. Éste planea seleccionar dinámicamente al mejor clasificador para cada entrada, así el atacante no podrá aprender el comportamiento de múltiples IDS Paya et al. (2024).

DESARROLLO

3.1. CRONOGRAMA DE ACTIVIDADES

En la figura 3.1 se muestra el calendario de actividades, seguido para el desarrollo del proyecto *Exploración Del Espacio Latente Para La Identificación De Anomalías En Redes*

ACTIVIDAD		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Investigación teórica y estado del arte: Para la correcta elaboración del proyecto fue necesario hacer un estudio a fondo acerca de las áreas como redes, IDEs y análisis de datos.	P																
	R																
Instalación del entorno y preparación: Se seleccionó una herramienta de trabajo para el desarrollo del código y se preparó el entorno.	P																
	R																
Primeras pruebas del funcionamiento normal del código K-means: Se solucionaron los problemas de compatibilidad para hacer funcionar el código original.	P																
	R																
Actualizaciones del código K-means de cada Gráfica: Las actualizaciones se dividieron en partes, donde cada una corresponde a una gráfica resultando 5 partes, donde se realizaron las mejoras y actualizaciones para su adecuado funcionamiento	P																
	R																
Optimización y reducción del código: Con el código funcionando correctamente se procedió a eliminar las partes innecesarias o a acortar algunas en las que fue posible.	P																
	R																
Pruebas, ejecución y análisis de resultados: Se hicieron las pruebas finales analizando cada gráfica para confirmar el correcto funcionamiento del código y se hicieron las correcciones en los casos que se necesitó.	P																
	R																
Documentación del reporte de residencia: Con el trabajo terminado y las anotaciones que se hicieron durante el proceso se desarrolló la documentación de todas las partes del proyecto.	P																
	R																
Correcciones, conclusiones y entrega final: Se reciben las correcciones por parte de los asesores y se hacen los cambios correspondientes para dar por terminado el proyecto.	P																
	R																
FECHAS DE SEGUIMIENTO		25/08/2025 -29/09/2025				29/09/2025 -10/11/2025				10/11/2025 -12/12/2025							

Figura 3.1: Calendario de actividades

3.2. Preparación del entorno de trabajo

3.2.1. Análisis de distintos IDE

Para la programación del código se hizo un análisis de distintos IDE para ocupar el que ofreciera mejores prestaciones para el entorno actual.

PyCharm: Es muy útil al auto completar código, pero se tiene muy poca experiencia en ese entorno.

Anaconda (Jupyter Notebooks): Se pensó ocupar por la practicidad de poder ejecutar el código por celdas pero al no poder encontrar compatibilidad adecuada con las librerías y versión de Python se optó por probar otra opción.

Visual Studio Code (VS Code): Gran herramienta con la mayor cantidad de plugins y extensiones de todos los IDE teniendo mucha practicidad, por lo tanto fue la herramienta definitiva y se programó dentro de ésta.

3.2.2. Creación de un entorno virtual

Es de suma importancia tener cuidado con la compatibilidad de las distintas versiones de Python. En el caso de la versión del sistema operativo llegó a dar conflictos de compatibilidad al igual que instalar otras versiones de Python en las variables de entorno. La solución ante este problema fue la creación de un entorno virtual.

Así que, se creó dentro de una carpeta específica con la ruta "*C : /Users/santi/Desktop/Kmeans*", dentro se instaló Python 3.11 y las librerías necesarias: PySpark, Numpy y Matplotlib con el comando "`pip install nombre_de_la_libreria`".

El uso de un entorno virtual nos permitió un mejor control de las librerías y evitar conflictos con las versiones que ya contaba el sistema operativo además de que facilita su reproducibilidad a futuro y en otros dispositivos.

El entorno se activa con el comando `.\.venv\Scripts\activate` y el código se ejecuta con `python nombre_del_archivo.py`.

3.2.3. Instalación y configuración de Python

Se optó por ocupar la versión más reciente, siendo Python 3.13 pensando que era la mejor opción pero no fue posible pues también tenía muchos problemas de

compatibilidad con las mismas librerías, esto sucede cuando una versión aún es muy nueva lo cual necesita un tiempo para que las librerías también se puedan actualizar a esta.

Se realizó una investigación acerca de cuál sería la versión ideal a usar por ello se decidió instalar Python 3.11 ya que es la versión de Python más reciente y estable para funcionar con múltiples librerías además de las ya mencionadas.

3.2.4. Exploración y comprensión de las características

Se hizo un estudio extenso del dataset KDDCUP99 que cuenta con 42 columnas que dan información acerca de cómo se comportan los ataques en una red. Entre las cuales algunas son: duration (tiempo de conexión en segundos), protocol (tipo de red utilizado como tcp, udp, icmp etc), servicio (http, ftp, etc), Src_bytes / Dst_bytes (corresponde al número de datos movidos), flag (denota el estado de la conexión que puede ser s0, reject entre otros), etc. Entre estos el más importante es type de tipo label, este nos dice que tipo de ataque es el que estamos manejando y analizando.

3.3. Código k-means

El código con el que se trabajó fue originalmente obtenido del libro “Hands-On Artificial Intelligence for Cybersecurity”.

Antes de comenzar a trabajar se probó y corrigieron algunos errores como la inicialización, pues al trabajar con un entorno virtual se necesita la versión de Python del entorno virtual y no la del sistema operativo. Se dejaron 10 gb de RAM en el entorno virtual para no tener problemas de velocidad como se muestra en la Figura 3.1.

Figura 3.1: Configuración para el entorno virtual

```
1 os.environ['PYSPARK-PYTHON'] = sys.executable
2 os.environ['PYSPARK-DRIVER-PYTHON'] = sys.executable
3 conf = SparkConf().setAppName("KMeansAnomalyDetection").
    setMaster("local[*]").set("spark.driver.memory", "10g")
```

3.4. Limpieza y procesamiento de los datos

3.4.1. Carga de KDDCUP99

El dataset kddcup99 se encuentra en el almacenamiento local, en este caso se encuentra en la carpeta. "C : /Users/santi/kddcup.data". En el código uno de los primeros pasos es usar la base de datos KDDCUP. La importación se realiza con las instrucciones mostradas en la Figura 3.2.

Figura 3.2: Se carga el dataset KDDCUP99

```
1 input\_path\_of\_file = "kddcup.data"
2 data\_raw = sc.textFile(input\_path\_of\_file, 12)
```

3.4.2. Eliminación de características innecesarias

Para poder usar de forma correcta, es necesario limpiar los datos, esto significa eliminar las características (columnas) que tienen datos str (cadenas), pues estas características no se pueden graficar fácilmente, las cuales son protocol_type, service, flag y label. Así que se crea un nuevo dataframe sin estos datos. Las instrucciones para limpiar los datos se muestran en la Figura 3.3.

Figura 3.3: Función de limpieza de vectores

```
1 def parseVector(line):
2     columns = line.split(',')
3     thelabel = columns[-1]
4     featurevector = columns[:-1]
5     featurevector = [element for i, element in enumerate(
6         featurevector) if i not in [1, 2, 3]]
7     featurevector = np.array(featurevector, dtype=np.float64)
8     return (thelabel, featurevector)
```

3.5. k-means con dos clústeres

La varianza es de suma importancia para saber cuáles valores representarán los ejes de la gráfica 3D.

Figura 3.4: k-means con dos clústers

```
1
```

```

2 def getFeatVecs(df_with_features, feature_column_names):
3     agg_expressions = [variance(c).alias(c) for c in
4                         feature_column_names]
5     variance_row = df_with_features.agg(*agg_expressions).first()
6     variances_list = [variance_row[c] for c in
                        feature_column_names]
7     return np.array(variances_list)

```

Se obtienen las varianzas de cada una de las 37 características restantes y en base a esas se hace una gráfica tridimensional tomando como ejes las 3 características con las varianzas más altas. El primer ciclo for itera sobre la lista de nombres de las características para calcular sus varianzas, posteriormente, en el segundo ciclo itera nuevamente para extraer los valores numéricos para crear una lista estándar de las varianzas. El código de este proceso se muestra en la Figura 3.4.

3.6. Desbalance de datos

Se obtiene la Suma de los Errores Cuadrados Dentro del Conjunto (WSSSE) del modelo entrenado `k_clusters` que sirve para medir la calidad numérica de los clústers. Entre más bajo sea el valor significa que los puntos están muy cerca del centroide, siendo esto lo que se busca.

Se toman los datos originales con la columna `label` a los cuales se les agrega la columna `prediction` donde se le asigna el cluster 0 o 1. Como se puede observar en la Figura 3.5.

Figura 3.5: Repartición de clústers

```

1 WSSSE = k_clusters.summary.trainingCost
2 print("Within Set Sum of Squared Error = " + str(WSSSE))
3
4 predictions_with_labels = k_clusters.transform(df_assembled.select
5     ("label", "features"))
6 clusterLabel = predictions_with_labels.groupBy("prediction", "
    label").count()

```


3.7. Método del codo para el k óptimo

Para obtener el valor del k óptimo se ocupó el método del codo, siendo el más apto para esta situación así que se grafican los distintos resultados que nos puede dar en intervalo de 5 a 126 con un valor de k de 20 en 20. De esta forma no hay que gastar tanto poder computacional en obtener más resultados que no son tan necesarios.

Se crean distintos modelos uno para cada punto que será graficado. Mientras es más alto sea el valor de WSSSE más alto se posicionará en la gráfica.

Figura 3.6: Entorno virtual

```

1 k_values = range(5, 126, 20)
2
3 def clustering_error_Score(thedata_df, k):
4     kmeans = KMeans(k=k, maxIter=10, initMode="random", seed=42)
5     model = kmeans.fit(thedata_df)
6     WSSSE = model.summary.trainingCost
7     return WSSSE

```

3.8. Método del codo con datos normalizados

La mejor forma para graficar el método del codo es haciendo que sean compatibles con ayuda de la normalización usando la estandarización z-score. Se usa la estandarización z-score para lograr una gráfica compatible con el análisis.

$$z = \frac{x - \mu}{\sigma} \quad (3.1)$$

Donde:

μ =Media de la columna σ =Desviación estándar

Figura 3.7: Calcular varianza

```

1 scaler = StandardScaler(inputCol="features",
2                           outputCol="scaledFeatures",
3                           withStd=True,
4                           withMean=True)
5
6 scalerModel = scaler.fit(thedata)

```

3.9. Modernización del código y Optimización de la Arquitectura de Datos

3.9.1. Migración de RDDs por dataframes

Una vez que el código funcionó correctamente se realizaron las actualizaciones del código que es pasar de RDDs (Resilient Distributed Datasets) por dataframes.

En la versión original el código se tenía que procesar línea por línea, por ejemplo, en `parseVector` pero cambiando a Dataframes ahora se crean arreglos que se procesan con `numpy`. Al usar un dataframe se aprovecha la capacidad que tienen para la gestión de grandes volúmenes de datos como los 4.9 millones de datos que usa KDDCUP99, facilitando procesos como el de filtrado. El uso de un entorno virtual que se agregó al código facilita replicar el proyecto, pues no es necesario hacer configuraciones complicadas y se evitan los problemas de compatibilidad.

3.9.2. Actualización de librerías

La librería más importante `pyspark.mllib` (RDDs) se migró a `pyspark.ml` para el uso de los dataframes que trajo consigo todas las ventajas anteriormente mencionadas.

Dentro de la librería se ocupan algunas de sus funciones como `StandardScaler` que reemplaza la función manual que se hacía con `normalize`. La información se encapsula con `SparkSession` lo cual facilita la configuración inicial del entorno. Para las operaciones matemáticas básicas se utiliza `StandardScaler` que lo hace de forma vectorizada en lugar de hacerlas línea por línea. Con `pyspark.sql.functions` las funciones se trabajan directamente en el motor de spark en lugar de convertir los datos de java a Python y viceversa.

Puede consultar el código completo que se ha elaborado en este proyecto dentro del apéndice A.

RESULTADOS

En este capítulo se presentan los resultados obtenidos del análisis del dataset KDDCUP99 mediante el algoritmo k-means, con el objetivo de explorar la estructura subyacente de los datos y evaluar su capacidad para distinguir comportamientos normales y anómalos en el dataset.

4.1. k-means con dos clústeres

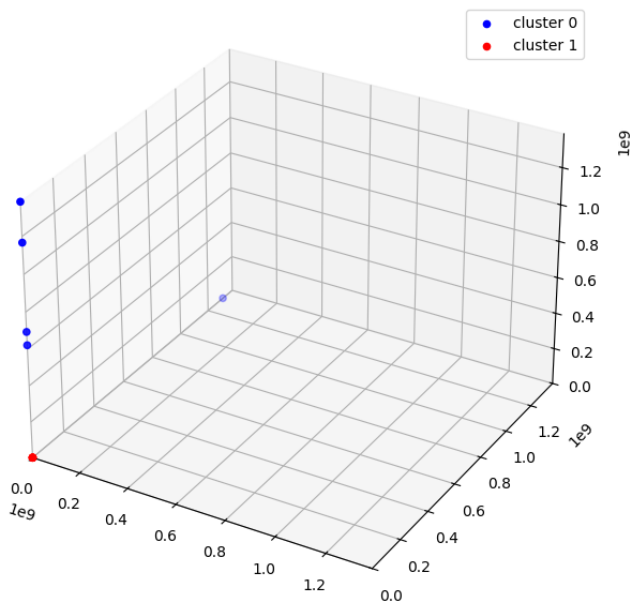


Figura 4.1: k-means con dos clústeres

En la figura 4.1 se muestra una representación tridimensional de un primer agrupamiento con $k = 2$. En cada uno de los 3 ejes tenemos las 3 características con la mayor varianza. Dentro están los dos clústeres el 0 con azul y 1 con el rojo, el fenómeno que se observa es la desproporción de los datos ya que la inmensa mayoría se encuentran encimados en el origen.

Por lo tanto, aún no se puede realizar una correcta interpretación por las diferencias de rangos que existen en los datos del dataset además de que esa es la razón por la que los ejes tienen números con cantidades de $1e9$ que probable-

mente son causados por características como `src_bytes` y `dst_bytes`.

4.2. Desbalance de datos

La tabla 4.1 muestra los 22 tipos de ataques y el comportamiento normal. Se observa su clúster perteneciente de cada ataque y en que cantidades están presentes dentro del dataset KDDCUP99.

Cluster	Ataque	Cantidad
0	portsweep	5
1	smurf	2807886
1	neptune	1072017
1	normal	972781
1	satan	15892
1	ipsweep	12481
1	portsweep	10408
1	nmap	2316
1	back	2203
1	warezclient	1020
1	teardrop	979
1	pod	264
1	guess_passwd	53
1	buffer_overflow	30
1	land	21
1	warezmaster	20
1	imap	12
1	rootkit	10
1	loadmodule	9
1	ftp_write	8
1	multihop	7
1	phf	4
1	perl	3
1	spy	2
Totales:	2	24
		4898431

Tabla 4.1: Listado de los 22 tipos de ataques incluidos el KDDCUP99

Como se puede observar en la Tabla 4.1, existe un desbalance completo en los clústeres pues solo hubo 5 ataques pertenecientes a "portsweep" que se colocaron en el clúster 0 sin una razón especial, mientras que el resto se colocó en el clúster 1.

4.3. Método del codo para el k óptimo

Para determinar el número óptimo de clústeres, se utilizó el método del codo. Para ello se graficaron algunos valores de k como se muestra en la Figura 4.2.

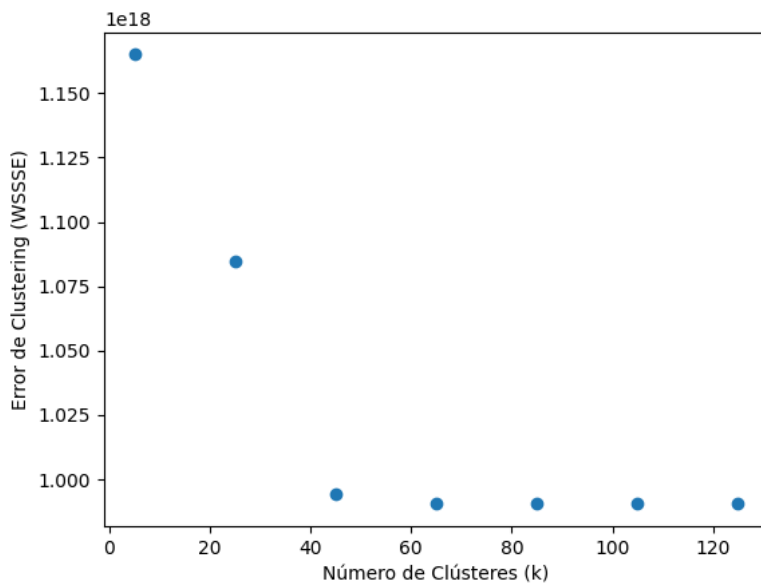


Figura 4.2: Método del codo con valores de k

Los valores de K van de 20 en 20 desde el 0 a 120 y se logra ver que se van estabilizando a partir de $k = 45$ pues el error WSSSE deja de disminuir en comparación a los valores superiores de K. Esta gráfica no se puede interpretar de forma definitiva debido a la falta de la normalización pero se puede comprender

a qué valores se apunta el valor de K.

4.4. Método del codo con datos normalizados

La gráfica del codo con los datos normalizados ya muestra datos más cambiantes por lo que ya se puede elegir un número óptimo de clústeres como se muestra en la Figura 4.3.

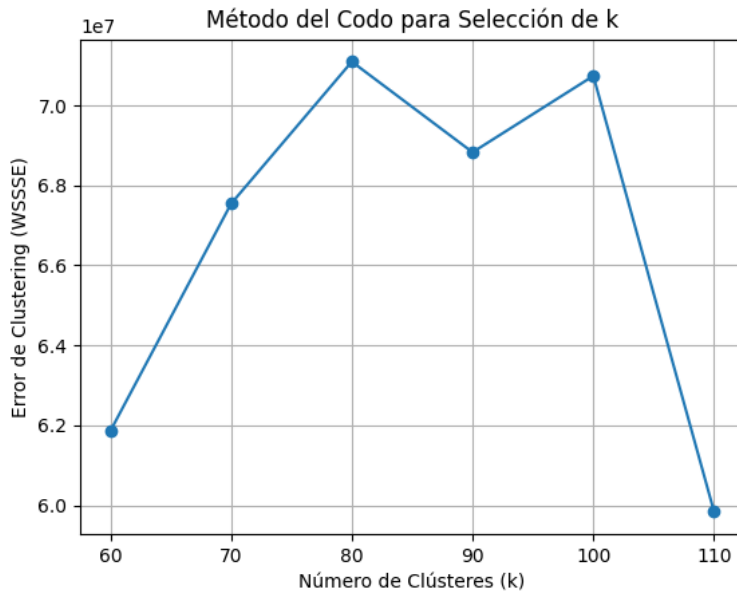


Figura 4.3: Método del codo con valores normalizados

Se puede visualizar que en la gráfica de la Figura 4.3 hay un punto de inflexión en 70 y 90 lo que significa que hay un equilibrio entre ese número de clústeres y el error WSSSE por lo que se tomarán 90 clústers.

4.5. Clústeres antes de la normalización

La gráfica 3D cuenta con 90 clústeres que se distinguen por ser puntos de distintos colores que están concentrados en el origen del gráfico como se muestra en la Figura 4.4.

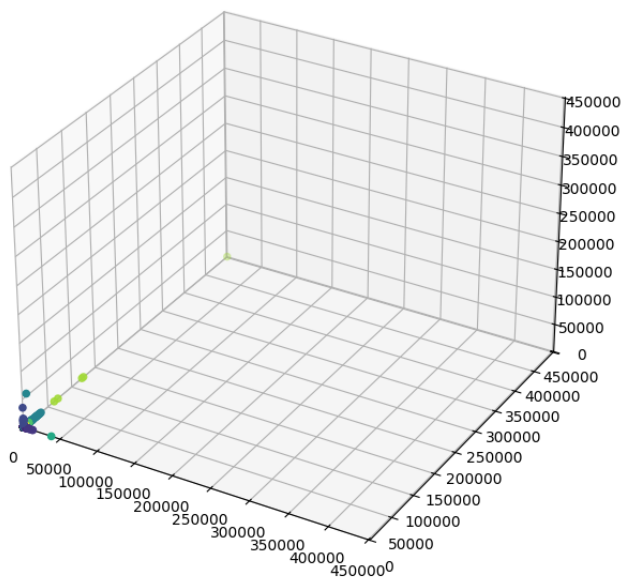


Figura 4.4: Clústeres antes de la normalización

Sin normalizar los datos, es muy complicado interpretar la gráfica, al estar todos los clústeres tan cerca ya que deben estar más separados para poder visualizar cuales tienen comportamientos anormales. Debido a la desproporción en las características (columnas del KDDCUP99), no se pueden comparar pequeños rangos con rangos grandes como lo son `src_bytes` y `dst_bytes` que llegan a tener valores con millones mientras hay varias características que solo manejan valores binarios como `land`, `logged_in`, entre otros.

4.6. Clústeres después de la normalización

Con la normalización se pueden observar muchos clústeres que se salen del origen además de que los ejes se manejan en rangos del 0 al 50 como se puede observar en la Figura 4.5.

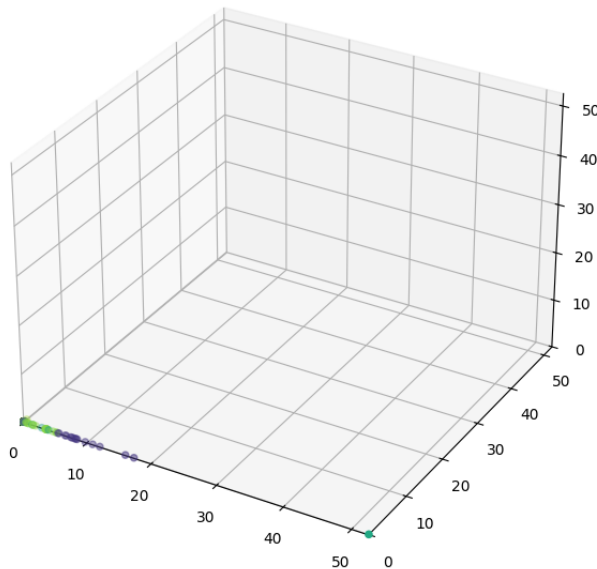


Figura 4.5: Clústeres después de la normalización

Gracias a la normalización podemos interpretar bien los valores de las gráficas ya que todos los valores se encuentran en los mismos rangos. Las anomalías son los clústeres que se alejan del origen donde están los comportamientos normales, es decir, las coordenadas (0,0), el resto son lo que podemos interpretar como los distintos tipos de ataques que se encuentran en el dataset.

CAPÍTULO 5

CONCLUSIONES

El desarrollo del presente proyecto abordó la migración del algoritmo k-means desde la librería `pyspark.mllib` hacia la API moderna `pyspark.ml`, lo cual permitió actualizar una implementación basada en RDDs a un enfoque fundamentado en DataFrames, más adecuado para el procesamiento de grandes volúmenes de datos. Esta transición resultó especialmente relevante para el análisis del dataset KDDCUP99, cuyo tamaño y complejidad demandan mecanismos de cómputo distribuidos eficientes.

Asimismo, se evidenció la importancia de determinar un número óptimo de clústeres, ya que este parámetro influye directamente en el equilibrio entre el uso de recursos computacionales y la capacidad del modelo para representar adecuadamente la estructura de los datos. El empleo del método del codo permitió establecer un criterio inicial para la selección de k , destacando la necesidad de un análisis previo antes de fijar dicho valor.

El estudio también puso de manifiesto la relevancia del preprocesamiento y la limpieza de los datos, dado que la ausencia de normalización impide una interpretación coherente de la distribución y separación de los clústeres. Esta limitación fue corregida mediante la aplicación de estandarización Z-score, la cual permitió homogeneizar las escalas de las características y mejorar sustancialmente la interpretabilidad de los resultados.

A partir de las visualizaciones obtenidas tras la normalización, fue posible identificar patrones de comportamiento diferenciados, donde los clústeres alejados del origen pueden interpretarse como anomalías de red, mientras que aquellos concentrados en torno al origen representan comportamientos normales. Este resultado confirma la viabilidad del enfoque no supervisado para la detección exploratoria de intrusiones.

Finalmente, el proyecto fue desarrollado y documentado de forma que garantiza su reproducibilidad, al haberse implementado en un entorno virtualizado y utilizando versiones actualizadas de las librerías y dependencias. Esto no solo facilita su mantenimiento y extensión futura, sino que también refuerza su valor como base técnica para trabajos posteriores en el área de detección de anomalías en ciberseguridad.

5.1. Recomendaciones y trabajo futuro

El análisis realizado sobre el dataset KDDCUP99 representa únicamente una etapa inicial del proyecto. Como trabajo futuro, se recomienda extender la metodología propuesta a datasets más recientes y representativos de escenarios actuales de tráfico de red, tales como UNSW-NB15 (2015) y CICIDS2017 (2017), con el fin de evaluar la robustez, generalización y vigencia del enfoque no supervisado basado en k-means frente a patrones de ataque más modernos y heterogéneos.

Asimismo, se identifica la necesidad de optimizar y refactorizar ciertos componentes del código para mejorar su eficiencia, modularidad y reutilización, facilitando su adaptación a distintos conjuntos de datos con características y distribuciones diversas. En particular, resulta pertinente incorporar mecanismos más flexibles de preprocesamiento, normalización y selección de características, así como explorar estrategias alternativas de evaluación y validación de clústeres. Estas mejoras permitirían fortalecer el pipeline y consolidarlo como una herramienta más general y escalable para el análisis de anomalías en ciberseguridad.

Referencias

- (2024). Anomaly and intrusion detection using deep learning for software-defined networks: A survey. *Expert Systems with Applications*, 256:124982.
- Akgun, D., Hizal, S., y Cavusoglu, U. (2022). A new ddos attacks intrusion detection model based on deep learning for cybersecurity. *Computers Security*, 118:102748.
- Aktar, S. y Yasin Nur, A. (2023). Towards ddos attack detection using deep learning approach. *Computers Security*, 129:103251.
- Bunmi, G. (2024). Identifying botnets within the traffic generated by a network in two. *International Journal of Scientific Research in Computer Science and Engineering*, p'aginas 77–93.
- Chou, E. (2018). *Distributed Denial of Service (DDoS) Practical Detection and Defense*. O'Reilly.
- Coronado, G. B. (2024). *Gestion de incidentes*. Tutor Formacion, San Milan.
- Diana, L., Dini, P., y Paolini, D. (2025). Overview on intrusion detection systems for computers networking security. *Computers*, 14(3).
- Frank, C. (2022). Cyber risk and cybersecurity: a systematic review of data. *Springer Nature Link*, p'agina 698–736.
- Hnamte, V. y Hussain, J. (2023). Dcnbnilstm: An efficient hybrid deep learning-based intrusion detection system. *Telematics and Informatics Reports*, 10:100053.
- ILLOWSKY, B. (2023). *Introductory Statistics 2e*. OpenStax, Houston.
- Louk, M. H. L. y Tama, B. A. (2023). Dual-ids: A bagging-based gradient boosting decision tree model for network anomaly intrusion detection system. *Expert Systems with Applications*, 213:119030.
- Mustapha, A., Khatoun, R., Zeadally, S., Chbib, F., Fadlallah, A., Fahs, W., y El Attar, A. (2023). Detecting ddos attacks using adversarial neural network. *Computers Security*, 127:103117.
- Paya, A., Arroni, S., García-Díaz, V., y Gómez, A. (2024). Apollon: A robust defense system against adversarial machine learning attacks in intrusion detection systems. *Computers Security*, 136:103546.
- Perez, F. C. (2023). *Probabilidad y estadística*. klik soluciones educativas, Ciudad de México.
- Sayed, M. S. E., Le-Khac, N.-A., Azer, M. A., y Jurcut, A. D. (2022). A flow-based anomaly detection approach with feature selection method against

- ddos attacks in sdns. *IEEE Transactions on Cognitive Communications and Networking*, 8(4):1862–1880.
- Sheikh, A. (2021). *Certified Ethical Hacker (CEH) Preparation Guide: Lesson-Based Review of Ethical Hacking and Penetration Testing*. Apress.
- Shi, S., Han, D., y Cui, M. (2023). A multimodal hybrid parallel network intrusion detection model. *Connection Science*, 35(1):2227780.
- Talukder, M. A., Islam, M., Uddin, M. A., Hasan, F., Sharmin, S., Alyami, S., y Moni, M. A. (2024). Machine learning-based network intrusion detection for big and imbalanced data using oversampling, stacking feature embedding and feature extraction. *Journal of Big Data*, 11.
- Vera, J. P. (2013). Propuesta de seguridad a sistemas unix para mitigar escalamiento de privilegios.
- Y.Mehelbei, V. P. (2022). Detection of attacks of the u2r category by means of the som on database nsl-kdd. *System technologies*, p'aginas 19–27.
- Youdi, G. (2023). A survey on dataset quality in machine learning. *Elsevier*, p'aginas 1–12.

CÓDIGO FUENTE DEL PROYECTO

```
1 import os
2 import sys
3 from pyspark.sql import SparkSession, Row
4 from pyspark.sql.functions import col, variance
5 from pyspark.ml.feature import VectorAssembler, StandardScaler
6 from pyspark.ml.clustering import KMeans
7 import matplotlib.pyplot as plt
8 import numpy as np
9 from operator import add
10
11 # CONFIGURACIÓN DE SPARK
12
13 os.environ['PYSPARK_PYTHON'] = sys.executable
14 os.environ['PYSPARK_DRIVER_PYTHON'] = sys.executable
15
16 spark = SparkSession.builder \
17     .appName("KMeansAnomalyDetection") \
18     .master("local[*]") \
19     .config("spark.driver.memory", "10g") \
20     .getOrCreate()
21
22 sc = spark.sparkContext
23 print("Spark Context iniciado")
24
25 # CARGA Y PREPROCESAMIENTO DE DATOS
26
27 input_path_of_file = "kddcup.data"
28 data_raw = sc.textFile(input_path_of_file, 12)
29
30
31 def parseVector(line):
32     columns = line.split(',')
33     thelabel = columns[-1]
34     featurevector = columns[:-1]
35     featurevector = [element for i, element in enumerate(
36         featurevector) if i not in [1, 2, 3]]
37     featurevector = np.array(featurevector, dtype=np.float64)
38     return (thelabel, featurevector)
```

```

38
39
40 labelsAndData_rdd = data_raw.map(parseVector).cache()
41
42 sample = labelsAndData_rdd.first()
43 num_features = len(sample[1])
44
45 feature_cols = [f"feature_{i}" for i in range(num_features)]
46
47 labelsAndData_rdd_lists = labelsAndData_rdd.map(lambda x: (x[0], x
48     [1].tolist()))
49
50 labelsAndData_df = labelsAndData_rdd_lists.map(
51     lambda x: Row(label=x[0], features_list=x[1])
52 ).toDF()
53
54 for i in range(num_features):
55     labelsAndData_df = labelsAndData_df.withColumn(
56         f"feature_{i}",
57         col("features_list")[i]
58     )
59
60 labelsAndData_df = labelsAndData_df.drop("features_list")
61 labelsAndData_df.cache()
62
63 assembler = VectorAssembler(inputCols=feature_cols, outputCol="
64     features")
65 df_assembled = assembler.transform(labelsAndData_df)
66
67 thedata = df_assembled.select("features").cache()
68 n = thedata.count()
69
70 # ENTRENAMIENTO INICIAL DE k-means CON 2 CLÚSTERS
71
72 kmeans = KMeans(k=2, maxIter=10, initMode="random", seed=42)
73 k_clusters = kmeans.fit(thedata)
74
75 def getFeatVecs(df_with_features, feature_column_names):
76     agg_expressions = [variance(c).alias(c) for c in
77         feature_column_names]
78     variance_row = df_with_features.agg(*agg_expressions).first()
79     variances_list = [variance_row[c] for c in
80         feature_column_names]
81     return np.array(variances_list)
82
83 vars_ = getFeatVecs(labelsAndData_df, feature_cols)

```



```

82 thedata_rdd = thedata.rdd.map(lambda row: row.features.toArray())
83 mean = thedata_rdd.map(lambda x: x[1]).reduce(add) / n
84 print(thedata_rdd.filter(lambda x: x[1] > 10*mean).count())
85
86
87 indices_of_variance = [t[0] for t in sorted(enumerate(vars_), key=
    lambda x: x[1])[-3:]]
88
89 predictions = k_clusters.transform(thedata)
90
91 rdd0 = predictions.filter(col("prediction") == 0).select("features
    ").rdd.map(lambda row: row.features.toArray())
92 rdd1 = predictions.filter(col("prediction") == 1).select("features
    ").rdd.map(lambda row: row.features.toArray())
93
94 cluster_0 = rdd0.take(5)
95 cluster_1 = rdd1.take(5)
96
97 cluster_0_projected = np.array([[point[i] for i in
    indices_of_variance] for point in cluster_0])
98 cluster_1_projected = np.array([[point[i] for i in
    indices_of_variance] for point in cluster_1])
99
100 M = max(max(cluster_1_projected.flatten()), max(
    cluster_0_projected.flatten()))
101 m = min(min(cluster_1_projected.flatten()), min(
    cluster_0_projected.flatten()))
102
103 fig2plot = plt.figure(figsize=(8, 8))
104 pltx = fig2plot.add_subplot(111, projection='3d')
105 pltx.scatter(cluster_0_projected[:, 0], cluster_0_projected[:, 1],
    cluster_0_projected[:, 2], c="b")
106 pltx.scatter(cluster_1_projected[:, 0], cluster_1_projected[:, 1],
    cluster_1_projected[:, 2], c="r")
107 pltx.set_xlim(m, M)
108 pltx.set_ylim(m, M)
109 pltx.set_zlim(m, M)
110 pltx.legend(["cluster_0", "cluster_1"])
111 plt.show()
112
113
114 def euclidean_distance_points(x1, x2):
115     x3 = x1 - x2
116     return np.sqrt(x3.T.dot(x3))
117
118
119 WSSSE = k_clusters.summary.trainingCost
120 print("Within Set Sum of Squared Error = " + str(WSSSE))

```

```

121
122 predictions_with_labels = k_clusters.transform(df_assembled.select
    ("label", "features"))
123 clusterLabel = predictions_with_labels.groupBy("prediction", "
    label").count()
124
125 for items in clusterLabel.collect():
126     print(items)
127
128
129 # MÉTODO DEL CODO - DATOS SIN NORMALIZAR
130
131 k_values = range(5, 126, 20)
132
133 def clustering_error_Score(thedata_df, k):
134     kmeans = KMeans(k=k, maxIter=10, initMode="random", seed=42)
135     model = kmeans.fit(thedata_df)
136     WSSSE = model.summary.trainingCost
137     return WSSSE
138
139
140 k_scores = [clustering_error_Score(thedata, k) for k in k_values]
141 for score in k_scores:
142     print(score)
143
144 plt.scatter(k_values, k_scores)
145 plt.xlabel('Número de Clústeres (k)')
146 plt.ylabel('Error de Clustering (WSSSE)')
147 plt.show()
148
149 # NORMALIZACIÓN DE DATOS
150
151 scaler = StandardScaler(inputCol="features",
152                          outputCol="scaledFeatures",
153                          withStd=True,
154                          withMean=True)
155
156 scalerModel = scaler.fit(thedata)
157
158 normalized = scalerModel.transform(thedata).select('scaledFeatures
    ')
159 normalized = normalized.withColumnRenamed('scaledFeatures', '
    features').cache()
160
161 print("Datos normalizados (2 ejemplos)")
162 print(normalized.take(2))
163
164 # MÉTODO DEL CODO - DATOS NORMALIZADOS

```

```
165 k_range = range(60, 111, 10)
166 k_scores = [clustering_error_Score(normalized, k) for k in k_range
167             ]
168
169 print("Valores de error")
170 for kscore in k_scores:
171     print(kscore)
172
173 plt.plot(k_range, k_scores, marker='o', linestyle='--')
174 plt.xlabel("Número de Clústeres (k)")
175 plt.ylabel("Error de Clustering (WSSSE)")
176 plt.title("Método del Codo para Selección de k")
177 plt.grid(True)
178 plt.show()
179
180 # VISUALIZACIÓN ANTES DE NORMALIZACION
181
182 K_norm = 90
183
184 var = getFeatVecs(labelsAndData_df, feature_cols)
185 indices_of_variance = [t[0] for t in sorted(enumerate(var), key=
186     lambda x: x[1])[-3:]]
187
188 dataprojected = thedata.randomSplit([1.0, 999.0])[0].cache()
189
190 kclusters = KMeans(k=K_norm, maxIter=10, initMode="random", seed
191     =42).fit(thedata)
192
193 listdataprojected = dataprojected.rdd.map(lambda row: row.features
194     .toArray()).collect()
195 projected_data = np.array([[point[i] for i in indices_of_variance]
196     for point in listdataprojected])
197
198 predictions_proj = kclusters.transform(dataprojected)
199 klabels = [row.prediction for row in predictions_proj.collect()]
200
201 Maxi = max(projected_data.flatten())
202 mini = min(projected_data.flatten())
203
204 figs = plt.figure(figsize=(8, 8))
205 pltx = figs.add_subplot(111, projection='3d')
206 pltx.scatter(projected_data[:, 0], projected_data[:, 1],
207     projected_data[:, 2], c=klabels)
208 pltx.set_xlim(mini, Maxi)
209 pltx.set_ylim(mini, Maxi)
210 pltx.set_zlim(mini, Maxi)
```

```

207 plt.set_title("Antes de la normalización")
208 plt.show()
209
210 # VISUALIZACIÓN DESPUÉS DE NORMALIZACION (GRÁFICA 5)
211
212 normalized_rdd = normalized.rdd.map(lambda row: row.features.
    toArray().toList())
213 normalized_with_cols = normalized_rdd.map(
214     lambda x: Row(**{f"feature_{i}": x[i] for i in range(len(x))})
215 ).toDF()
216
217 var_normalized = getFeatVecs(normalized_with_cols, feature_cols)
218 indices_of_variance_norm = [t[0] for t in sorted(enumerate(
    var_normalized), key=lambda x: x[1])[-3:]]
219
220 kclusters = KMeans(k=K_norm, maxIter=10, initMode="random", seed
    =42).fit(normalized)
221
222 dataprojected_normed = scalerModel.transform(dataprojected).select
    ('scaledFeatures')
223 dataprojected_normed = dataprojected_normed.withColumnRenamed('
    scaledFeatures', 'features').cache()
224
225 dataprojected_normed_list = dataprojected_normed.rdd.map(lambda
    row: row.features.toArray()).collect()
226 projected_data = np.array([[point[i] for i in indices_of_variance]
    for point in dataprojected_normed_list])
227
228 predictions_normed = kclusters.transform(dataprojected_normed)
229 klabels = [row.prediction for row in predictions_normed.collect()]
230
231 Maxi = max(projected_data.flatten())
232 mini = min(projected_data.flatten())
233
234 figs = plt.figure(figsize=(8, 8))
235 pltx = figs.add_subplot(111, projection='3d')
236 pltx.scatter(projected_data[:, 0], projected_data[:, 1],
    projected_data[:, 2], c=klabels)
237 pltx.set_xlim(mini, Maxi)
238 pltx.set_ylim(mini, Maxi)
239 pltx.set_zlim(mini, Maxi)
240 pltx.set_title("Después de la normalización")
241
242 print(f"Puntos en dataprojected: {dataprojected.count()}")
243
244 plt.show() # Gráfica 5

```